## 7.1 The List ADT

In Chapter 6, we introduced the stack, queue, and deque abstract data types, and discussed how either an array or a linked list could be used for storage in an efficient concrete implementation of each. Each of those ADTs represents a linearly ordered sequence of elements. The deque is the most general of the three, yet even so, it only allows insertions and deletions at the front or back of a sequence.

In this chapter, we explore several abstract data types that represent a linear sequence of elements, but with more general support for adding or removing elements at arbitrary positions. However, designing a single abstraction that is well suited for efficient implementation with either an array or a linked list is challenging, given the very different nature of these two fundamental data structures.

Locations within an array are easily described with an integer ***index***. Recall that an index of an element $e$ in a sequence is equal to the number of elements before $e$ in that sequence. By this definition, the first element of a sequence has index 0, and the last has index $n-1$, assuming that $n$ denotes the total number of elements. The notion of an element's index is well defined for a linked list as well, although we will see that it is not as convenient of a notion, as there is no way to efficiently access an element at a given index without traversing a portion of the linked list that depends upon the magnitude of the index.

With that said, Java defines a general interface, `java.util.List`, that includes the following index-based methods (and more):

`size()`: Returns the number of elements in the list.

`isEmpty()`: Returns a boolean indicating whether the list is empty.

`get(`$i$`)`: Returns the element of the list having index $i$; an error condition occurs if $i$ is not in range $[0, \text{size}()-1]$.

`set(`$i$`, `$e$`)`: Replaces the element at index $i$ with $e$, and returns the old element that was replaced; an error condition occurs if $i$ is not in range $[0, \text{size}()-1]$.

`add(`$i$`, `$e$`)`: Inserts a new element $e$ into the list so that it has index $i$, moving all subsequent elements one index later in the list; an error condition occurs if $i$ is not in range $[0, \text{size}()]$.

`remove(`$i$`)`: Removes and returns the element at index $i$, moving all subsequent elements one index earlier in the list; an error condition occurs if $i$ is not in range $[0, \text{size}()-1]$.

We note that the index of an existing element may change over time, as other elements are added or removed in front of it. We also draw attention to the fact that the range of valid indices for the `add` method includes the current size of the list, in which case the new element becomes the last.

www.EBooksWorld.ir

Example 7.1 demonstrates a series of operations on a list instance, and Code Fragment 7.1 below provides a formal definition of our simplified version of the List interface; we use an IndexOutOfBoundsException to signal an invalid index argument.

**Example 7.1:** *We demonstrate operations on an initially empty list of characters.*

| Method | Return Value | List Contents |
|---|---|---|
| add(0, A) | – | (A) |
| add(0, B) | – | (B, A) |
| get(1) | A | (B, A) |
| set(2, C) | "error" | (B, A) |
| add(2, C) | – | (B, A, C) |
| add(4, D) | "error" | (B, A, C) |
| remove(1) | A | (B, C) |
| add(1, D) | – | (B, D, C) |
| add(1, E) | – | (B, E, D, C) |
| get(4) | "error" | (B, E, D, C) |
| add(4, F) | – | (B, E, D, C, F) |
| set(2, G) | D | (B, E, G, C, F) |
| get(2) | G | (B, E, G, C, F) |

```
1  /** A simplified version of the java.util.List interface. */
2  public interface List<E> {
3    /** Returns the number of elements in this list. */
4    int size( );
5
6    /** Returns whether the list is empty. */
7    boolean isEmpty( );
8
9    /** Returns (but does not remove) the element at index i. */
10   E get(int i) throws IndexOutOfBoundsException;
11
12   /** Replaces the element at index i with e, and returns the replaced element. */
13   E set(int i, E e) throws IndexOutOfBoundsException;
14
15   /** Inserts element e to be at index i, shifting all subsequent elements later. */
16   void add(int i, E e) throws IndexOutOfBoundsException;
17
18   /** Removes/returns the element at index i, shifting subsequent elements earlier. */
19   E remove(int i) throws IndexOutOfBoundsException;
20 }
```

**Code Fragment 7.1:** A simple version of the List interface.

www.EBooksWorld.ir

## 7.2 Array Lists

An obvious choice for implementing the list ADT is to use an array $A$, where $A[i]$ stores (a reference to) the element with index $i$. We will begin by assuming that we have a fixed-capacity array, but in Section 7.2.1 describe a more advanced technique that effectively allows an array-based list to have unbounded capacity. Such an unbounded list is known as an ***array list*** in Java (or a ***vector*** in C++ and in the earliest versions of Java).

With a representation based on an array $A$, the get($i$) and set($i$, $e$) methods are easy to implement by accessing $A[i]$ (assuming $i$ is a legitimate index). Methods add($i$, $e$) and remove($i$) are more time consuming, as they require shifting elements up or down to maintain our rule of always storing an element whose list index is $i$ at index $i$ of the array. (See Figure 7.1.) Our initial implementation of the ArrayList class follows in Code Fragments 7.2 and 7.3.

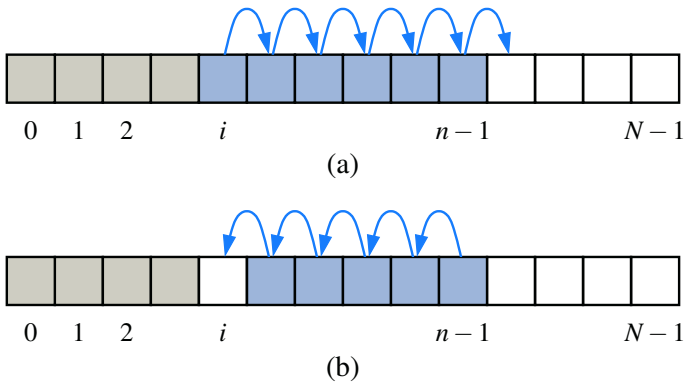



**Figure 7.1:** Array-based implementation of an array list that is storing $n$ elements: (a) shifting up for an insertion at index $i$; (b) shifting down for a removal at index $i$.

```
1  public class ArrayList<E> implements List<E> {
2    // instance variables
3    public static final int CAPACITY=16;      // default array capacity
4    private E[ ] data;                        // generic array used for storage
5    private int size = 0;                     // current number of elements
6    // constructors
7    public ArrayList( ) { this(CAPACITY); }   // constructs list with default capacity
8    public ArrayList(int capacity) {          // constructs list with given capacity
9      data = (E[ ]) new Object[capacity];     // safe cast; compiler may give warning
10   }
```

**Code Fragment 7.2:** An implementation of a simple ArrayList class with bounded capacity. (Continues in Code Fragment 7.3.)

www.EBooksWorld.ir

```
11    // public methods
12    /** Returns the number of elements in the array list. */
13    public int size( ) { return size; }
14    /** Returns whether the array list is empty. */
15    public boolean isEmpty( ) { return size == 0; }
16    /** Returns (but does not remove) the element at index i. */
17    public E get(int i) throws IndexOutOfBoundsException {
18      checkIndex(i, size);
19      return data[i];
20    }
21    /** Replaces the element at index i with e, and returns the replaced element. */
22    public E set(int i, E e) throws IndexOutOfBoundsException {
23      checkIndex(i, size);
24      E temp = data[i];
25      data[i] = e;
26      return temp;
27    }
28    /** Inserts element e to be at index i, shifting all subsequent elements later. */
29    public void add(int i, E e) throws IndexOutOfBoundsException,
30                                                IllegalStateException {
31      checkIndex(i, size + 1);
32      if (size == data.length)                  // not enough capacity
33        throw new IllegalStateException("Array is full");
34      for (int k=size-1; k >= i; k--)           // start by shifting rightmost
35        data[k+1] = data[k];
36      data[i] = e;                              // ready to place the new element
37      size++;
38    }
39    /** Removes/returns the element at index i, shifting subsequent elements earlier. */
40    public E remove(int i) throws IndexOutOfBoundsException {
41      checkIndex(i, size);
42      E temp = data[i];
43      for (int k=i; k < size-1; k++)            // shift elements to fill hole
44        data[k] = data[k+1];
45      data[size-1] = null;                      // help garbage collection
46      size--;
47      return temp;
48    }
49    // utility method
50    /** Checks whether the given index is in the range [0, n-1]. */
51    protected void checkIndex(int i, int n) throws IndexOutOfBoundsException {
52      if (i < 0 || i >= n)
53        throw new IndexOutOfBoundsException("Illegal index: " + i);
54    }
55  }
```

**Code Fragment 7.3:** An implementation of a simple ArrayList class with bounded capacity. (Continued from Code Fragment 7.2.)

www.EBooksWorld.ir

### The Performance of a Simple Array-Based Implementation

Table 7.1 shows the worst-case running times of the methods of an array list with $n$ elements realized by means of an array. Methods isEmpty, size, get and set clearly run in $O(1)$ time, but the insertion and removal methods can take much longer than this. In particular, add$(i, e)$ runs in time $O(n)$. Indeed, the worst case for this operation occurs when $i$ is 0, since all the existing $n$ elements have to be shifted forward. A similar argument applies to method remove$(i)$, which runs in $O(n)$ time, because we have to shift backward $n-1$ elements in the worst case, when $i$ is 0. In fact, assuming that each possible index is equally likely to be passed as an argument to these operations, their average running time is $O(n)$, for we will have to shift $n/2$ elements on average.

| Method | Running Time |
|---|---|
| size( ) | $O(1)$ |
| isEmpty( ) | $O(1)$ |
| get$(i)$ | $O(1)$ |
| set$(i, e)$ | $O(1)$ |
| add$(i, e)$ | $O(n)$ |
| remove$(i)$ | $O(n)$ |

**Table 7.1:** Performance of an array list with $n$ elements realized by a fixed-capacity array.

Looking more closely at add$(i, e)$ and remove$(i)$, we note that they each run in time $O(n-i+1)$, for only those elements at index $i$ and higher have to be shifted up or down. Thus, inserting or removing an item at the end of an array list, using the methods add$(n, e)$ and remove$(n-1)$ respectively, takes $O(1)$ time each. Moreover, this observation has an interesting consequence for the adaptation of the array list ADT to the deque ADT from Section 6.3.1. If we do the "obvious" thing and store elements of a deque so that the first element is at index 0 and the last element at index $n-1$, then methods addLast and removeLast of the deque each run in $O(1)$ time. However, methods addFirst and removeFirst of the deque each run in $O(n)$ time.

Actually, with a little effort, we can produce an array-based implementation of the array list ADT that achieves $O(1)$ time for insertions and removals at index 0, as well as insertions and removals at the end of the array list. Achieving this requires that we give up on our rule that an element at index $i$ is stored in the array at index $i$, however, as we would have to use a circular array approach like the one we used in Section 6.2 to implement a queue. We leave the details of this implementation as Exercise C-7.25.

www.EBooksWorld.ir

### 7.2.1 Dynamic Arrays

The ArrayList implementation in Code Fragments 7.2 and 7.3 (as well as those for a stack, queue, and deque from Chapter 6) has a serious limitation; it requires that a fixed maximum capacity be declared, throwing an exception if attempting to add an element once full. This is a major weakness, because if a user is unsure of the maximum size that will be reached for a collection, there is risk that either too large of an array will be requested, causing an inefficient waste of memory, or that too small of an array will be requested, causing a fatal error when exhausting that capacity.

Java's ArrayList class provides a more robust abstraction, allowing a user to add elements to the list, with no apparent limit on the overall capacity. To provide this abstraction, Java relies on an algorithmic sleight of hand that is known as a ***dynamic array***.

In reality, elements of an ArrayList are stored in a traditional array, and the precise size of that traditional array must be internally declared in order for the system to properly allocate a consecutive piece of memory for its storage. For example, Figure 7.2 displays an array with 12 cells that might be stored in memory locations 2146 through 2157 on a computer system.

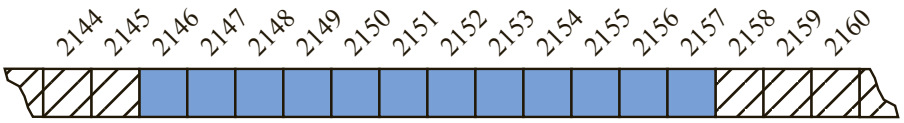



**Figure 7.2:** An array of 12 cells, allocated in memory locations 2146 through 2157.

Because the system may allocate neighboring memory locations to store other data, the capacity of an array cannot be increased by expanding into subsequent cells.

The first key to providing the semantics of an unbounded array is that an array list instance maintains an internal array that often has greater capacity than the current length of the list. For example, while a user may have created a list with five elements, the system may have reserved an underlying array capable of storing eight object references (rather than only five). This extra capacity makes it easy to add a new element to the end of the list by using the next available cell of the array.

If a user continues to add elements to a list, all reserved capacity in the underlying array will eventually be exhausted. In that case, the class requests a new, larger array from the system, and copies all references from the smaller array into the beginning of the new array. At that point in time, the old array is no longer needed, so it can be reclaimed by the system. Intuitively, this strategy is much like that of the hermit crab, which moves into a larger shell when it outgrows its previous one.

www.EBooksWorld.ir

### 7.2.2 Implementing a Dynamic Array

We now demonstrate how our original version of the ArrayList, from Code Fragments 7.2 and 7.3, can be transformed to a dynamic-array implementation, having unbounded capacity. We rely on the same internal representation, with a traditional array $A$, that is initialized either to a default capacity or to one specified as a parameter to the constructor.

The key is to provide means to "grow" the array $A$, when more space is needed. Of course, we cannot actually grow that array, as its capacity is fixed. Instead, when a call to add a new element risks ***overflowing*** the current array, we perform the following additional steps:

1. Allocate a new array $B$ with larger capacity.
2. Set $B[k] = A[k]$, for $k = 0, \ldots, n-1$, where $n$ denotes current number of items.
3. Set $A = B$, that is, we henceforth use the new array to support the list.
4. Insert the new element in the new array.

An illustration of this process is shown in Figure 7.3.

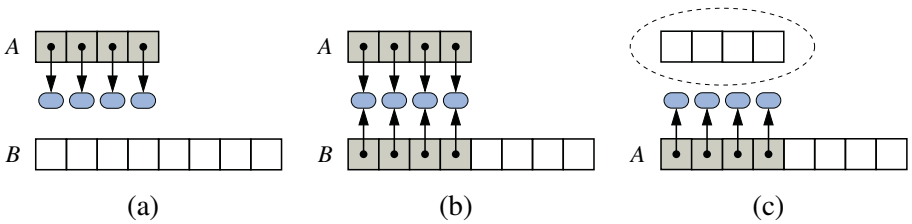



**Figure 7.3:** An illustration of "growing" a dynamic array: (a) create new array $B$; (b) store elements of $A$ in $B$; (c) reassign reference $A$ to the new array. Not shown is the future garbage collection of the old array, or the insertion of a new element.

Code Fragment 7.4 provides a concrete implementation of a resize method, which should be included as a protected method within the original ArrayList class. The instance variable data corresponds to array $A$ in the above discussion, and local variable temp corresponds to array $B$.

```
/** Resizes internal array to have given capacity >= size. */
protected void resize(int capacity) {
  E[ ] temp = (E[ ]) new Object[capacity]; // safe cast; compiler may give warning
  for (int k=0; k < size; k++)
    temp[k] = data[k];
  data = temp;                             // start using the new array
}
```

**Code Fragment 7.4:** An implementation of the ArrayList.resize method.

www.EBooksWorld.ir

The remaining issue to consider is how large of a new array to create. A commonly used rule is for the new array to have twice the capacity of the existing array that has been filled. In Section 7.2.3, we will provide a mathematical analysis to justify such a choice.

To complete the revision to our original ArrayList implementation, we redesign the add method so that it calls the new resize utility when detecting that the current array is filled (rather than throwing an exception). The revised version appears in Code Fragment 7.5.

```
28   /** Inserts element e to be at index i, shifting all subsequent elements later. */
29   public void add(int i, E e) throws IndexOutOfBoundsException {
30     checkIndex(i, size + 1);
31     if (size == data.length)                  // not enough capacity
32       resize(2 * data.length);                // so double the current capacity
...    // rest of method unchanged...
```

**Code Fragment 7.5:** A revision to the ArrayList.add method, originally from Code Fragment 7.3, which calls the resize method of Code Fragment 7.4 when more capacity is needed.

Finally, we note that our original implementation of the ArrayList class includes two constructors: a default constructor that uses an initial capacity of 16, and a parameterized constructor that allows the caller to specify a capacity value. With the use of dynamic arrays, that capacity is no longer a fixed limit. Still, greater efficiency is achieved when a user selects an initial capacity that matches the actual size of a data set, as this can avoid time spent on intermediate array reallocations and potential space that is wasted by having too large of an array.

## 7.2.3 Amortized Analysis of Dynamic Arrays

In this section, we will perform a detailed analysis of the running time of operations on dynamic arrays. As a shorthand notation, let us refer to the insertion of an element to be the last element in an array list as a ***push*** operation.

The strategy of replacing an array with a new, larger array might at first seem slow, because a single push operation may require $\Omega(n)$ time to perform, where $n$ is the current number of elements in the array. (Recall, from Section 4.3.1, that big-Omega notation, describes an asymptotic lower bound on the running time of an algorithm.) However, by doubling the capacity during an array replacement, our new array allows us to add $n$ further elements before the array must be replaced again. In this way, there are many simple push operations for each expensive one (see Figure 7.4). This fact allows us to show that a series of push operations on an initially empty dynamic array is efficient in terms of its total running time.

www.EBooksWorld.ir

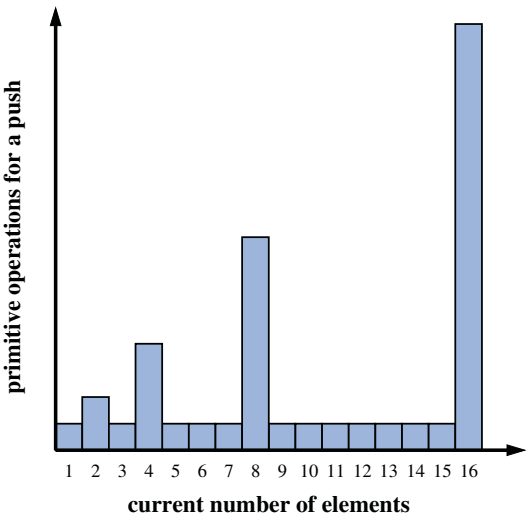



**Figure 7.4:** Running times of a series of push operations on a dynamic array.

Using an algorithmic design pattern called ***amortization***, we show that performing a sequence of push operations on a dynamic array is actually quite efficient. To perform an ***amortized analysis***, we use an accounting technique where we view the computer as a coin-operated appliance that requires the payment of one ***cyber-dollar*** for a constant amount of computing time. When an operation is executed, we should have enough cyber-dollars available in our current "bank account" to pay for that operation's running time. Thus, the total amount of cyber-dollars spent for any computation will be proportional to the total time spent on that computation. The beauty of using this analysis method is that we can overcharge some operations in order to save up cyber-dollars to pay for others.

**Proposition 7.2:** *Let $L$ be an initially empty array list with capacity one, implemented by means of a dynamic array that doubles in size when full. The total time to perform a series of $n$ push operations in $L$ is $O(n)$.*

**Justification:** Let us assume that one cyber-dollar is enough to pay for the execution of each push operation in $L$, excluding the time spent for growing the array. Also, let us assume that growing the array from size $k$ to size $2k$ requires $k$ cyber-dollars for the time spent initializing the new array. We shall charge each push operation three cyber-dollars. Thus, we overcharge each push operation that does not cause an overflow by two cyber-dollars. Think of the two cyber-dollars profited in an insertion that does not grow the array as being "stored" with the cell in which the element was inserted. An overflow occurs when the array $L$ has $2^i$ elements, for some integer $i \geq 0$, and the size of the array used by the array representing $L$ is $2^i$. Thus, doubling the size of the array will require $2^i$ cyber-dollars. Fortunately, these cyber-dollars can be found stored in cells $2^{i-1}$ through $2^i - 1$. (See Figure 7.5.)

www.EBooksWorld.ir

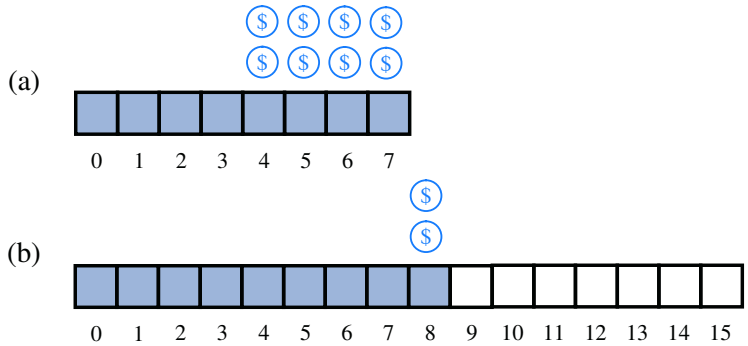



**Figure 7.5:** Illustration of a series of push operations on a dynamic array: (a) an 8-cell array is full, with two cyber-dollars "stored" at cells 4 through 7; (b) a push operation causes an overflow and a doubling of capacity. Copying the eight old elements to the new array is paid for by the cyber-dollars already stored in the table. Inserting the new element is paid for by one of the cyber-dollars charged to the current push operation, and the two cyber-dollars profited are stored at cell 8.

Note that the previous overflow occurred when the number of elements became larger than $2^{i-1}$ for the first time, and thus the cyber-dollars stored in cells $2^{i-1}$ through $2^i - 1$ have not yet been spent. Therefore, we have a valid amortization scheme in which each operation is charged three cyber-dollars and all the computing time is paid for. That is, we can pay for the execution of $n$ push operations using $3n$ cyber-dollars. In other words, the amortized running time of each push operation is $O(1)$; hence, the total running time of $n$ push operations is $O(n)$. ■

### Geometric Increase in Capacity

Although the proof of Proposition 7.2 relies on the array being doubled each time it is expanded, the $O(1)$ amortized bound per operation can be proven for any geometrically increasing progression of array sizes. (See Section 2.2.3 for discussion of geometric progressions.) When choosing the geometric base, there exists a trade-off between runtime efficiency and memory usage. If the last insertion causes a resize event, with a base of 2 (i.e., doubling the array), the array essentially ends up twice as large as it needs to be. If we instead increase the array by only 25% of its current size (i.e., a geometric base of 1.25), we do not risk wasting as much memory in the end, but there will be more intermediate resize events along the way. Still it is possible to prove an $O(1)$ amortized bound, using a constant factor greater than the 3 cyber-dollars per operation used in the proof of Proposition 7.2 (see Exercise R-7.7). The key to the performance is that the amount of additional space is proportional to the current size of the array.

www.EBooksWorld.ir

### Beware of Arithmetic Progression

To avoid reserving too much space at once, it might be tempting to implement a dynamic array with a strategy in which a constant number of additional cells are reserved each time an array is resized. Unfortunately, the overall performance of such a strategy is significantly worse. At an extreme, an increase of only one cell causes each push operation to resize the array, leading to a familiar $1+2+3+\cdots+n$ summation and $\Omega(n^2)$ overall cost. Using increases of 2 or 3 at a time is slightly better, as portrayed in Figure 7.4, but the overall cost remains quadratic.

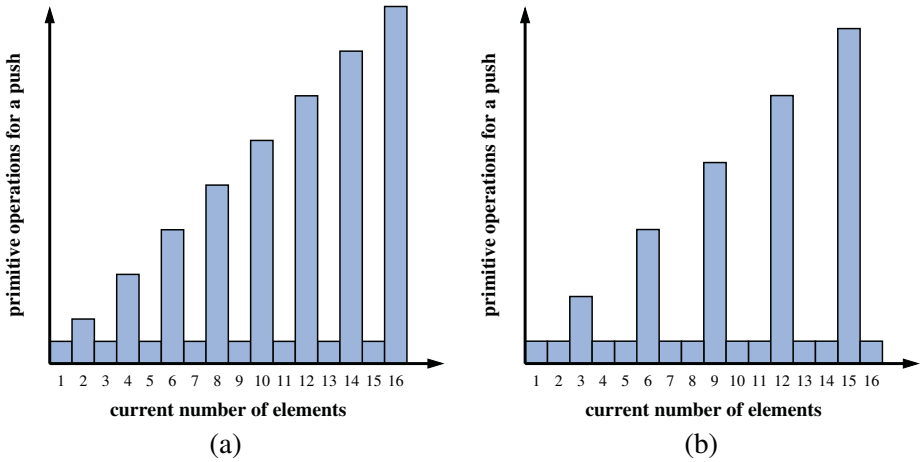



**Figure 7.6:** Running times of a series of push operations on a dynamic array using arithmetic progression of sizes. Part (a) assumes an increase of 2 in the size of the array, while part (b) assumes an increase of 3.

Using a *fixed* increment for each resize, and thus an arithmetic progression of intermediate array sizes, results in an overall time that is quadratic in the number of operations, as shown in the following proposition. In essence, even an increase in 10,000 cells per resize will become insignificant for large data sets.

**Proposition 7.3:** *Performing a series of $n$ push operations on an initially empty dynamic array using a fixed increment with each resize takes $\Omega(n^2)$ time.*

**Justification:** Let $c > 0$ represent the fixed increment in capacity that is used for each resize event. During the series of $n$ push operations, time will have been spent initializing arrays of size $c, 2c, 3c, \ldots, mc$ for $m = \lceil n/c \rceil$, and therefore, the overall time is proportional to $c+2c+3c+\cdots+mc$. By Proposition 4.3, this sum is

$$\sum_{i=1}^{m} ci = c \cdot \sum_{i=1}^{m} i = c\frac{m(m+1)}{2} \geq c\frac{\frac{n}{c}(\frac{n}{c}+1)}{2} \geq \frac{1}{2c} \cdot n^2.$$

Therefore, performing the $n$ push operations takes $\Omega(n^2)$ time. ■

www.EBooksWorld.ir

### Memory Usage and Shrinking an Array

Another consequence of the rule of a geometric increase in capacity when adding to a dynamic array is that the final array size is guaranteed to be proportional to the overall number of elements. That is, the data structure uses $O(n)$ memory. This is a very desirable property for a data structure.

If a container, such as an array list, provides operations that cause the removal of one or more elements, greater care must be taken to ensure that a dynamic array guarantees $O(n)$ memory usage. The risk is that repeated insertions may cause the underlying array to grow arbitrarily large, and that there will no longer be a proportional relationship between the actual number of elements and the array capacity after many elements are removed.

A robust implementation of such a data structure will shrink the underlying array, on occasion, while maintaining the $O(1)$ amortized bound on individual operations. However, care must be taken to ensure that the structure cannot rapidly oscillate between growing and shrinking the underlying array, in which case the amortized bound would not be achieved. In Exercise C-7.29, we explore a strategy in which the array capacity is halved whenever the number of actual element falls below one-fourth of that capacity, thereby guaranteeing that the array capacity is at most four times the number of elements; we explore the amortized analysis of such a strategy in Exercises C-7.30 and C-7.31.

## 7.2.4 Java's StringBuilder class

Near the beginning of Chapter 4, we described an experiment in which we compared two algorithms for composing a long string (Code Fragment 4.2). The first of those relied on repeated concatenation using the String class, and the second relied on use of Java's StringBuilder class. We observed the StringBuilder was significantly faster, with empirical evidence that suggested a quadratic running time for the algorithm with repeated concatenations, and a linear running time for the algorithm with the StringBuilder. We are now able to explain the theoretical underpinning for those observations.

The StringBuilder class represents a mutable string by storing characters in a dynamic array. With analysis similar to Proposition 7.2, it guarantees that a series of append operations resulting in a string of length $n$ execute in a combined time of $O(n)$. (Insertions at positions other than the end of a string builder do not carry this guarantee, just as they do not for an ArrayList.)

In contrast, the repeated use of string concatenation requires quadratic time. We originally analyzed that algorithm on page 172 of Chapter 4. In effect, that approach is akin to a dynamic array with an arithmetic progression of size one, repeatedly copying all characters from one array to a new array with size one greater than before.

www.EBooksWorld.ir

## 7.3 Positional Lists

When working with array-based sequences, integer indices provide an excellent means for describing the location of an element, or the location at which an insertion or deletion should take place. However, numeric indices are not a good choice for describing positions within a linked list because, knowing only an element's index, the only way to reach it is to traverse the list incrementally from its beginning or end, counting elements along the way.

Furthermore, indices are not a good abstraction for describing a more local view of a position in a sequence, because the index of an entry changes over time due to insertions or deletions that happen earlier in the sequence. For example, it may not be convenient to describe the location of a person waiting in line based on the index, as that requires knowledge of precisely how far away that person is from the front of the line. We prefer an abstraction, as characterized in Figure 7.7, in which there is some other means for describing a position.

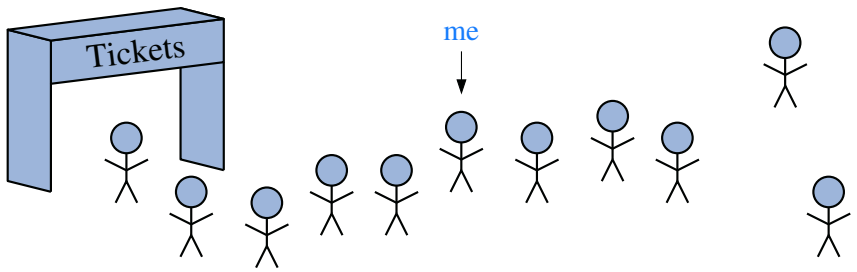



**Figure 7.7:** We wish to be able to identify the position of an element in a sequence without the use of an integer index. The label "me" represents some abstraction that identifies the position.

Our goal is to design an abstract data type that provides a user a way to refer to elements anywhere in a sequence, and to perform arbitrary insertions and deletions. This would allow us to efficiently describe actions such as a person deciding to leave the line before reaching the front, or allowing a friend to "cut" into line right behind him or her.

As another example, a text document can be viewed as a long sequence of characters. A word processor uses the abstraction of a ***cursor*** to describe a position within the document without explicit use of an integer index, allowing operations such as "delete the character at the cursor" or "insert a new character just after the cursor." Furthermore, we may be able to refer to an inherent position within a document, such as the beginning of a particular chapter, without relying on a character index (or even a chapter number) that may change as the document evolves.

www.EBooksWorld.ir

For these reasons, we temporarily forego the index-based methods of Java's formal List interface, and instead develop our own abstract data type that we denote as a ***positional list***. Although a positional list is an abstraction, and need not rely on a linked list for its implementation, we certainly have a linked list in mind as we design the ADT, ensuring that it takes best advantage of particular capabilities of a linked list, such as $O(1)$-time insertions and deletions at arbitrary positions (something that is not possible with an array-based sequence).

We face an immediate challenge in designing the ADT; to achieve constant time insertions and deletions at arbitrary locations, we effectively need a reference to the node at which an element is stored. It is therefore very tempting to develop an ADT in which a node reference serves as the mechanism for describing a position. In fact, our DoublyLinkedList class of Section 3.4.1 has methods addBetween and remove that accept node references as parameters; however, we intentionally declared those methods as private.

Unfortunately, the public use of nodes in the ADT would violate the object-oriented design principles of abstraction and encapsulation, which were introduced in Chapter 2. There are several reasons to prefer that we encapsulate the nodes of a linked list, for both our sake and for the benefit of users of our abstraction:

- It will be simpler for users of our data structure if they are not bothered with unnecessary details of our implementation, such as low-level manipulation of nodes, or our reliance on the use of sentinel nodes. Notice that to use the addBetween method of our DoublyLinkedList class to add a node at the beginning of a sequence, the header sentinel must be sent as a parameter.
- We can provide a more robust data structure if we do not permit users to directly access or manipulate the nodes. We can then ensure that users do not invalidate the consistency of a list by mismanaging the linking of nodes. A more subtle problem arises if a user were allowed to call the addBetween or remove method of our DoublyLinkedList class, sending a node that does not belong to the given list as a parameter. (Go back and look at that code and see why it causes a problem!)
- By better encapsulating the internal details of our implementation, we have greater flexibility to redesign the data structure and improve its performance. In fact, with a well-designed abstraction, we can provide a notion of a nonnumeric position, even if using an array-based sequence. (See Exercise C-7.43.)

Therefore, in defining the positional list ADT, we also introduce the concept of a ***position***, which formalizes the intuitive notion of the "location" of an element relative to others in the list. (When we do use a linked list for the implementation, we will later see how we can privately use node references as natural manifestations of positions.)

www.EBooksWorld.ir

### 7.3.1 Positions

To provide a general abstraction for the location of an element within a structure, we define a simple ***position*** abstract data type. A position supports the following single method:

getElement( ): Returns the element stored at this position.

A position acts as a marker or token within a broader positional list. A position $p$, which is associated with some element $e$ in a list $L$, does not change, even if the index of $e$ changes in $L$ due to insertions or deletions elsewhere in the list. Nor does position $p$ change if we replace the element $e$ stored at $p$ with another element. The only way in which a position becomes invalid is if that position (and its element) are explicitly removed from the list.

Having a formal definition of a position type allows positions to serve as parameters to some methods and return values from other methods of the positional list ADT, which we next describe.

### 7.3.2 The Positional List Abstract Data Type

We now view a ***positional list*** as a collection of positions, each of which stores an element. The accessor methods provided by the positional list ADT include the following, for a list $L$:

first( ): Returns the position of the first element of $L$ (or null if empty).

last( ): Returns the position of the last element of $L$ (or null if empty).

before($p$): Returns the position of $L$ immediately before position $p$ (or null if $p$ is the first position).

after($p$): Returns the position of $L$ immediately after position $p$ (or null if $p$ is the last position).

isEmpty( ): Returns true if list $L$ does not contain any elements.

size( ): Returns the number of elements in list $L$.

An error occurs if a position $p$, sent as a parameter to a method, is not a valid position for the list.

Note well that the first( ) and last( ) methods of the positional list ADT return the associated *positions*, not the *elements*. (This is in contrast to the corresponding first and last methods of the deque ADT.) The first element of a positional list can be determined by subsequently invoking the getElement method on that position, as first( ).getElement. The advantage of receiving a position as a return value is that we can subsequently use that position to traverse the list.

www.EBooksWorld.ir

As a demonstration of a typical traversal of a positional list, Code Fragment 7.6 traverses a list, named guests, that stores string elements, and prints each element while traversing from the beginning of the list to the end.

```
1  Position<String> cursor = guests.first( );
2  while (cursor != null) {
3    System.out.println(cursor.getElement( ));
4    cursor = guests.after(cursor);              // advance to the next position (if any)
5  }
```

**Code Fragment 7.6:** A traversal of a positional list.

This code relies on the convention that the null reference is returned when the after method is called upon the last position. (That return value is clearly distinguishable from any legitimate position.) The positional list ADT similarly indicates that the null value is returned when the before method is invoked at the front of the list, or when first or last methods are called upon an empty list. Therefore, the above code fragment works correctly even if the guests list is empty.

### Updated Methods of a Positional List

The positional list ADT also includes the following ***update*** methods:

addFirst($e$): Inserts a new element $e$ at the front of the list, returning the position of the new element.

addLast($e$): Inserts a new element $e$ at the back of the list, returning the position of the new element.

addBefore($p$, $e$): Inserts a new element $e$ in the list, just before position $p$, returning the position of the new element.

addAfter($p$, $e$): Inserts a new element $e$ in the list, just after position $p$, returning the position of the new element.

set($p$, $e$): Replaces the element at position $p$ with element $e$, returning the element formerly at position $p$.

remove($p$): Removes and returns the element at position $p$ in the list, invalidating the position.

There may at first seem to be redundancy in the above repertoire of operations for the positional list ADT, since we can perform operation addFirst($e$) with addBefore(first( ), $e$), and operation addLast($e$) with addAfter(last( ), $e$). But these substitutions can only be done for a nonempty list.

www.EBooksWorld.ir

**Example 7.4:** *The following table shows a series of operations on an initially empty positional list storing integers. To identify position instances, we use variables such as $p$ and $q$. For ease of exposition, when displaying the list contents, we use subscript notation to denote the position storing an element.*

| Method | Return Value | List Contents |
|---|---|---|
| addLast(8) | $p$ | $(8_p)$ |
| first( ) | $p$ | $(8_p)$ |
| addAfter($p$, 5) | $q$ | $(8_p, 5_q)$ |
| before($q$) | $p$ | $(8_p, 5_q)$ |
| addBefore($q$, 3) | $r$ | $(8_p, 3_r, 5_q)$ |
| $r$.getElement( ) | 3 | $(8_p, 3_r, 5_q)$ |
| after($p$) | $r$ | $(8_p, 3_r, 5_q)$ |
| before($p$) | null | $(8_p, 3_r, 5_q)$ |
| addFirst(9) | $s$ | $(9_s, 8_p, 3_r, 5_q)$ |
| remove(last( )) | 5 | $(9_s, 8_p, 3_r)$ |
| set($p$, 7) | 8 | $(9_s, 7_p, 3_r)$ |
| remove($q$) | "error" | $(9_s, 7_p, 3_r)$ |

## Java Interface Definitions

We are now ready to formalize the position ADT and positional list ADT. A Java Position interface, representing the position ADT, is given in Code Fragment 7.7. Following that, Code Fragment 7.8 presents a Java definition for our PositionalList interface. If the getElement( ) method is called on a Position instance that has previously been removed from its list, an IllegalStateException is thrown. If an invalid Position instance is sent as a parameter to a method of a PositionalList, an IllegalArgumentException is thrown. (Both of those exception types are defined in the standard Java hierarchy.)

```
1  public interface Position<E> {
2    /**
3     * Returns the element stored at this position.
4     *
5     * @return the stored element
6     * @throws IllegalStateException if position no longer valid
7     */
8    E getElement( ) throws IllegalStateException;
9  }
```

**Code Fragment 7.7:** The Position interface.

www.EBooksWorld.ir

```
1  /** An interface for positional lists. */
2  public interface PositionalList<E> {
3
4    /** Returns the number of elements in the list. */
5    int size( );
6
7    /** Tests whether the list is empty. */
8    boolean isEmpty( );
9
10   /** Returns the first Position in the list (or null, if empty). */
11   Position<E> first( );
12
13   /** Returns the last Position in the list (or null, if empty). */
14   Position<E> last( );
15
16   /** Returns the Position immediately before Position p (or null, if p is first). */
17   Position<E> before(Position<E> p) throws IllegalArgumentException;
18
19   /** Returns the Position immediately after Position p (or null, if p is last). */
20   Position<E> after(Position<E> p) throws IllegalArgumentException;
21
22   /** Inserts element e at the front of the list and returns its new Position. */
23   Position<E> addFirst(E e);
24
25   /** Inserts element e at the back of the list and returns its new Position. */
26   Position<E> addLast(E e);
27
28   /** Inserts element e immediately before Position p and returns its new Position. */
29   Position<E> addBefore(Position<E> p, E e)
30     throws IllegalArgumentException;
31
32   /** Inserts element e immediately after Position p and returns its new Position. */
33   Position<E> addAfter(Position<E> p, E e)
34     throws IllegalArgumentException;
35
36   /** Replaces the element stored at Position p and returns the replaced element. */
37   E set(Position<E> p, E e) throws IllegalArgumentException;
38
39   /** Removes the element stored at Position p and returns it (invalidating p). */
40   E remove(Position<E> p) throws IllegalArgumentException;
41 }
```

**Code Fragment 7.8:** The PositionalList interface.

www.EBooksWorld.ir

### 7.3.3 Doubly Linked List Implementation

Not surprisingly, our preferred implementation of the PositionalList interface relies on a doubly linked list. Although we implemented a DoublyLinkedList class in Chapter 3, that class does not adhere to the PositionalList interface.

In this section, we develop a concrete implementation of the PositionalList interface using a doubly linked list. The low-level details of our new linked-list representation, such as the use of header and trailer sentinels, will be identical to our earlier version; we refer the reader to Section 3.4 for a discussion of the doubly linked list operations. What differs in this section is our management of the positional abstraction.

The obvious way to identify locations within a linked list are node references. Therefore, we declare the nested Node class of our linked list so as to implement the Position interface, supporting the required getElement method. So the nodes *are* the positions. Yet, the Node class is declared as private, to maintain proper encapsulation. All of the public methods of the positional list rely on the Position type, so although we know we are sending and receiving nodes, these are only known to be positions from the outside; as a result, users of our class cannot call any method other than getElement( ).

In Code Fragments 7.9–7.12, we define a LinkedPositionalList class, which implements the positional list ADT. We provide the following guide to that code:

- Code Fragment 7.9 contains the definition of the nested Node<E> class, which implements the Position<E> interface. Following that are the declaration of the instance variables of the outer LinkedPositionalList class and its constructor.
- Code Fragment 7.10 begins with two important utility methods that help us robustly cast between the Position and Node types. The validate(p) method is called anytime the user sends a Position instance as a parameter. It throws an exception if it determines that the position is invalid, and otherwise returns that instance, implicitly cast as a Node, so that methods of the Node class can subsequently be called. The private position(node) method is used when about to return a Position to the user. Its primary purpose is to make sure that we do not expose either sentinel node to a caller, returning a **null** reference in such a case. We rely on both of these private utility methods in the public accessor methods that follow.
- Code Fragment 7.11 provides most of the public update methods, relying on a private addBetween method to unify the implementations of the various insertion operations.
- Code Fragment 7.12 provides the public remove method. Note that it sets all fields of the removed node back to null—a condition we can later detect to recognize a defunct position.

www.EBooksWorld.ir

```
1  /** Implementation of a positional list stored as a doubly linked list. */
2  public class LinkedPositionalList<E> implements PositionalList<E> {
3    //---------- nested Node class ----------
4    private static class Node<E> implements Position<E> {
5      private E element;                  // reference to the element stored at this node
6      private Node<E> prev;               // reference to the previous node in the list
7      private Node<E> next;               // reference to the subsequent node in the list
8      public Node(E e, Node<E> p, Node<E> n) {
9        element = e;
10       prev = p;
11       next = n;
12     }
13     public E getElement( ) throws IllegalStateException {
14       if (next == null)                          // convention for defunct node
15         throw new IllegalStateException("Position no longer valid");
16       return element;
17     }
18     public Node<E> getPrev( ) {
19       return prev;
20     }
21     public Node<E> getNext( ) {
22       return next;
23     }
24     public void setElement(E e) {
25       element = e;
26     }
27     public void setPrev(Node<E> p) {
28       prev = p;
29     }
30     public void setNext(Node<E> n) {
31       next = n;
32     }
33   } //---------- end of nested Node class ----------
34
35   // instance variables of the LinkedPositionalList
36   private Node<E> header;                       // header sentinel
37   private Node<E> trailer;                      // trailer sentinel
38   private int size = 0;                         // number of elements in the list
39
40   /** Constructs a new empty list. */
41   public LinkedPositionalList( ) {
42     header = new Node<>(null, null, null);      // create header
43     trailer = new Node<>(null, header, null);   // trailer is preceded by header
44     header.setNext(trailer);                    // header is followed by trailer
45   }
```

**Code Fragment 7.9:** An implementation of the LinkedPositionalList class. (Continues in Code Fragments 7.10–7.12.)

www.EBooksWorld.ir

```
46    // private utilities
47    /** Validates the position and returns it as a node. */
48    private Node<E> validate(Position<E> p) throws IllegalArgumentException {
49      if (!(p instanceof Node)) throw new IllegalArgumentException("Invalid p");
50      Node<E> node = (Node<E>) p;     // safe cast
51      if (node.getNext( ) == null)       // convention for defunct node
52        throw new IllegalArgumentException("p is no longer in the list");
53      return node;
54    }
55
56    /** Returns the given node as a Position (or null, if it is a sentinel). */
57    private Position<E> position(Node<E> node) {
58      if (node == header || node == trailer)
59        return null;     // do not expose user to the sentinels
60      return node;
61    }
62
63    // public accessor methods
64    /** Returns the number of elements in the linked list. */
65    public int size( ) { return size; }
66
67    /** Tests whether the linked list is empty. */
68    public boolean isEmpty( ) { return size == 0; }
69
70    /** Returns the first Position in the linked list (or null, if empty). */
71    public Position<E> first( ) {
72      return position(header.getNext( ));
73    }
74
75    /** Returns the last Position in the linked list (or null, if empty). */
76    public Position<E> last( ) {
77      return position(trailer.getPrev( ));
78    }
79
80    /** Returns the Position immediately before Position p (or null, if p is first). */
81    public Position<E> before(Position<E> p) throws IllegalArgumentException {
82      Node<E> node = validate(p);
83      return position(node.getPrev( ));
84    }
85
86    /** Returns the Position immediately after Position p (or null, if p is last). */
87    public Position<E> after(Position<E> p) throws IllegalArgumentException {
88      Node<E> node = validate(p);
89      return position(node.getNext( ));
90    }
```

**Code Fragment 7.10:** An implementation of the LinkedPositionalList class. (Continued from Code Fragment 7.9; continues in Code Fragments 7.11 and 7.12.)

www.EBooksWorld.ir

```
91    // private utilities
92    /** Adds element e to the linked list between the given nodes. */
93    private Position<E> addBetween(E e, Node<E> pred, Node<E> succ) {
94      Node<E> newest = new Node<>(e, pred, succ);  // create and link a new node
95      pred.setNext(newest);
96      succ.setPrev(newest);
97      size++;
98      return newest;
99    }
100
101   // public update methods
102   /** Inserts element e at the front of the linked list and returns its new Position. */
103   public Position<E> addFirst(E e) {
104     return addBetween(e, header, header.getNext( ));          // just after the header
105   }
106
107   /** Inserts element e at the back of the linked list and returns its new Position. */
108   public Position<E> addLast(E e) {
109     return addBetween(e, trailer.getPrev( ), trailer);          // just before the trailer
110   }
111
112   /** Inserts element e immediately before Position p, and returns its new Position.*/
113   public Position<E> addBefore(Position<E> p, E e)
114                                       throws IllegalArgumentException {
115     Node<E> node = validate(p);
116     return addBetween(e, node.getPrev( ), node);
117   }
118
119   /** Inserts element e immediately after Position p, and returns its new Position. */
120   public Position<E> addAfter(Position<E> p, E e)
121                                       throws IllegalArgumentException {
122     Node<E> node = validate(p);
123     return addBetween(e, node, node.getNext( ));
124   }
125
126   /** Replaces the element stored at Position p and returns the replaced element. */
127   public E set(Position<E> p, E e) throws IllegalArgumentException {
128     Node<E> node = validate(p);
129     E answer = node.getElement( );
130     node.setElement(e);
131     return answer;
132   }
```

**Code Fragment 7.11:** An implementation of the LinkedPositionalList class. (Continued from Code Fragments 7.9 and 7.10; continues in Code Fragment 7.12.)

www.EBooksWorld.ir

```
133   /** Removes the element stored at Position p and returns it (invalidating p). */
134   public E remove(Position<E> p) throws IllegalArgumentException {
135     Node<E> node = validate(p);
136     Node<E> predecessor = node.getPrev( );
137     Node<E> successor = node.getNext( );
138     predecessor.setNext(successor);
139     successor.setPrev(predecessor);
140     size−−;
141     E answer = node.getElement( );
142     node.setElement(null);                    // help with garbage collection
143     node.setNext(null);                       // and convention for defunct node
144     node.setPrev(null);
145     return answer;
146   }
147 }
```

**Code Fragment 7.12:** An implementation of the LinkedPositionalList class. (Continued from Code Fragments 7.9–7.11.)

## The Performance of a Linked Positional List

The positional list ADT is ideally suited for implementation with a doubly linked list, as all operations run in worst-case constant time, as shown in Table 7.2. This is in stark contrast to the ArrayList structure (analyzed in Table 7.1), which requires linear time for insertions or deletions at arbitrary positions, due to the need for a loop to shift other elements.

Of course, our positional list does not support the index-based methods of the official List interface of Section 7.1. It is possible to add support for those methods by traversing the list while counting nodes (see Exercise C-7.38), but that requires time proportional to the sublist that is traversed.

| Method | Running Time |
|---:|---|
| size( ) | $O(1)$ |
| isEmpty( ) | $O(1)$ |
| first( ), last( ) | $O(1)$ |
| before($p$), after($p$) | $O(1)$ |
| addFirst($e$), addLast($e$) | $O(1)$ |
| addBefore($p$, $e$), addAfter($p$, $e$) | $O(1)$ |
| set($p$, $e$) | $O(1)$ |
| remove($p$) | $O(1)$ |

**Table 7.2:** Performance of a positional list with $n$ elements realized by a doubly linked list. The space usage is $O(n)$.

www.EBooksWorld.ir

### Implementing a Positional List with an Array

We can implement a positional list $L$ using an array $A$ for storage, but some care is necessary in designing objects that will serve as positions. At first glance, it would seem that a position $p$ need only store the index $i$ at which its associated element is stored within the array. We can then implement method getElement($p$) simply by returning $A[i]$. The problem with this approach is that the index of an element $e$ changes when other insertions or deletions occur before it. If we have already returned a position $p$ associated with element $e$ that stores an outdated index $i$ to a user, the wrong array cell would be accessed when the position was used. (Remember that positions in a positional list should always be defined relative to their neighboring positions, not their indices.)

Hence, if we are going to implement a positional list with an array, we need a different approach. We recommend the following representation. Instead of storing the elements of $L$ directly in array $A$, we store a new kind of position object in each cell of $A$. A position $p$ stores the element $e$ as well as the current index $i$ of that element within the list. Such a data structure is illustrated in Figure 7.8.

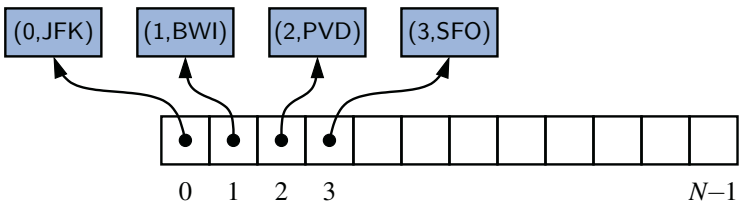



**Figure 7.8:** An array-based representation of a positional list.

With this representation, we can determine the index currently associated with a position, and we can determine the position currently associated with a specific index. We can therefore implement an accessor, such as before($p$), by finding the index of the given position and using the array to find the neighboring position.

When an element is inserted or deleted somewhere in the list, we can loop through the array to update the index variable stored in all later positions in the list that are shifted during the update.

### Efficiency Trade-Offs with an Array-Based Sequence

In this array implementation of a sequence, the addFirst, addBefore, addAfter, and remove methods take $O(n)$ time, because we have to shift position objects to make room for the new position or to fill in the hole created by the removal of the old position (just as in the insert and remove methods based on index). All the other position-based methods take $O(1)$ time.

www.EBooksWorld.ir

## 7.4 Iterators

An ***iterator*** is a software design pattern that abstracts the process of scanning through a sequence of elements, one element at a time. The underlying elements might be stored in a container class, streaming through a network, or generated by a series of computations.

In order to unify the treatment and syntax for iterating objects in a way that is independent from a specific organization, Java defines the java.util.Iterator interface with the following two methods:

hasNext( ): Returns true if there is at least one additional element in the sequence, and false otherwise.

next( ): Returns the next element in the sequence.

The interface uses Java's generic framework, with the next( ) method returning a parameterized element type. For example, the Scanner class (described in Section 1.6) formally implements the Iterator<String> interface, with its next( ) method returning a String instance.

If the next( ) method of an iterator is called when no further elements are available, a NoSuchElementException is thrown. Of course, the hasNext( ) method can be used to detect that condition before calling next( ).

The combination of these two methods allows a general loop construct for processing elements of the iterator. For example, if we let variable, iter, denote an instance of the Iterator<String> type, then we can write the following:

```
while (iter.hasNext( )) {
  String value = iter.next( );
  System.out.println(value);
}
```

The java.util.Iterator interface contains a third method, which is *optionally* supported by some iterators:

remove( ): Removes from the collection the element returned by the most recent call to next( ). Throws an IllegalStateException if next has not yet been called, or if remove was already called since the most recent call to next.

This method can be used to filter a collection of elements, for example to discard all negative numbers from a data set.

For the sake of simplicity, we will not implement the remove method for most data structures in this book, but we will give two tangible examples later in this section. If removal is not supported, an UnsupportedOperationException is conventionally thrown.

www.EBooksWorld.ir